
Simulando Qubits com PySCF

Uma Jornada do Zero aos Pontos Quânticos
para a Computação Quântica

Do primeiro `import pyscf` ao qubit de spin em Si:P

Cristiano Alves

Edição de Trabalho — Volume I

28 de julho de 2026

A modelagem computacional de *qubits* baseados em pontos quânticos deixou de ser uma curiosidade acadêmica para se tornar uma competência estratégica. Governos e empresas investem bilhões em computação quântica, e há uma demanda crescente por profissionais capazes de descrever, com rigor atômico, os dispositivos que hão de sustentar essa tecnologia. No entanto, quem tenta ingressar na área costuma esbarrar em dois obstáculos: pacotes de estrutura eletrônica com curvas de aprendizado íngremes e, muitas vezes, com custos de licenciamento proibitivos.

O PySCF — *Python-based Simulations of Chemistry Framework* — derruba essas barreiras. É livre, gratuito e integralmente programável em Python, a linguagem que se tornou a *lingua franca* da ciência computacional. Ainda assim, mesmo uma ferramenta amigável pode intimidar quem nunca executou um cálculo de química quântica ou escreveu uma linha de código.

Este livro nasce de uma convicção simples: **é possível partir do zero absoluto e chegar à simulação completa de um *qubit* real**. Nós o faremos com uma abordagem prática, na qual cada conceito é apresentado por meio de exemplos executáveis. Mais do que isso, todo o material converge para um **projeto longitudinal único** — a construção, passo a passo, de um modelo de *qubit* de spin em silício dopado com fósforo (Si:P). Você começará digitando `import pyscf` e terminará extraindo o acoplamento hiperfino, o tensor g , a energia de troca entre *qubits* e estimativas de tempos de descoerência.

Este primeiro volume cobre a **Parte 0 — Preparando o Laboratório Computacional**. O objetivo é claro: garantir que, ao virar a última página do Capítulo 1, você tenha um ambiente Python funcional, domine os elementos da linguagem indispensáveis à química quântica e esteja pronto para instalar e executar o PySCF. Nada será pressuposto; tudo será construído.

Cristiano Alves

Como Usar Este Livro

Este livro foi projetado para ser lido *com o computador ligado*. Cada trecho de código é curto o bastante para ser digitado à mão — e recomendamos que você o digite, pois o aprendizado se consolida pelos dedos, não apenas pelos olhos. Ao longo do texto você encontrará elementos visuais recorrentes:

Nota

Aprofunda um conceito, esclarece uma sutileza ou conecta o assunto a capítulos futuros. Ler é recomendável, mas o fluxo principal não depende disso.

Dica

Truques práticos, atalhos e boas práticas que economizam tempo e evitam frustrações comuns.

Atenção

Erros frequentes e armadilhas. Preste atenção especial: quase todo iniciante tropeça aqui pelo menos uma vez.

Mãos à obra

Exercícios guiados de digitar-e-executar. É onde a teoria vira prática. Não pule estas seções.

Projeto Âncora

Sinaliza um avanço no **Projeto Âncora**, o fio condutor que atravessa todo o livro. Cada etapa acrescenta uma peça ao seu *pipeline* de simulação de *qubits*.

Convenções tipográficas. Nomes de comandos, funções e trechos curtos de código aparecem em fonte monoespaçada. Blocos de código Python são numerados à esquerda e destacados em fundo cinza. Saídas de terminal aparecem em fundo escuro.

Dica

Todo o código deste livro estará disponível em um repositório on-line, em forma de *notebooks* Jupyter prontos para executar. Ainda assim, digite os exemplos ao menos uma vez: a fluência não se baixa, se constrói.

Prefácio	i
Como Usar Este Livro	ii
1 O Primeiro Cálculo com PySCF	1
1.1 A Anatomia de um Cálculo em PySCF	1
1.2 Definindo uma Molécula: o Objeto <code>Mole</code>	2
1.2.1 A geometria: o argumento <code>atom</code>	2
1.2.2 Inspeccionando o objeto <code>Mole</code>	2
1.3 O Que é uma Base?	3
1.3.1 Famílias de bases	3
1.3.2 O efeito da base na prática	4
1.4 Executando Hartree-Fock: o Objeto <code>SCF</code>	4
1.5 Interpretando a Saída	5
1.5.1 Energias orbitais, ocupações e o <i>gap</i>	6
1.6 Mãos à Obra: do Átomo de Hidrogênio à Molécula H_2	7
Resumo do Capítulo	8
Exercícios	9

CAPÍTULO 1

O Primeiro Cálculo com PySCF

No capítulo anterior montamos o laboratório: temos Python, NumPy e o PySCF instalados e verificados. Agora vamos usá-los para o que realmente importa — resolver a equação de Schrödinger eletrônica de um sistema real e obter sua energia. Ao final deste capítulo, você terá calculado, do zero, a energia do átomo de hidrogênio, da molécula H_2 e — como primeiro passo concreto do Projeto Âncora — do átomo de silício.

Não se preocupe se os fundamentos teóricos do método Hartree-Fock ainda parecerem nebulosos: o Capítulo 4 os desenvolverá em detalhe. Aqui, o objetivo é outro e igualmente essencial — **ganhar fluência no fluxo de trabalho do PySCF**: definir um sistema, escolher um método, executar o cálculo e interpretar a saída. Esse quarteto se repetirá, praticamente inalterado, em todos os cálculos do livro.

1.1 A Anatomia de um Cálculo em PySCF

Quase todo cálculo de estrutura eletrônica no PySCF segue três passos:

1. **Definir o sistema** — que átomos, em que posições, com que carga e spin, e em qual conjunto de funções de base. Isso é encapsulado no objeto `Mole` (de *molecule*).
2. **Escolher o método** — Hartree-Fock, DFT, pós-Hartree-Fock. Aqui usaremos o objeto `SCF` na sua forma Hartree-Fock.
3. **Executar e analisar** — disparar o cálculo com `kernel()` e ler os resultados: energia total, energias orbitais, convergência.

Em código, esse esqueleto cabe em cinco linhas:

Listing 1.1: O esqueleto de qualquer cálculo PySCF.

```
1 from pyscf import gto, scf          # 1. importa os modulos
2
3 mol = gto.M(atom='H 0 0 0; H 0 0 0.74', # 2. define o sistema
4       basis='sto-3g')
5
6 mf = scf.RHF(mol)                   # 3. escolhe o metodo (Hartree-Fock)
7 energia = mf.kernel()               # 4. executa e obtem a energia total
```

O módulo `gto` (*Gaussian-type orbitals*) cuida da definição do sistema; o módulo `scf` (*self-consistent field*) cuida dos métodos de campo autoconsistente, dos quais o Hartree-Fock é o mais fundamental. Vamos dissecar cada passo.

1.2 Definindo uma Molécula: o Objeto `Mole`

Tudo começa pela descrição do sistema físico. A função `gto.M()` constrói um objeto `Mole` que reúne toda a informação: geometria, base, carga e spin. Seus argumentos mais importantes são:

Listing 1.2: Os argumentos essenciais de `gto.M`.

```
1 from pyscf import gto
2
3 mol = gto.M(
4     atom='''H 0.0 0.0 0.0
5           H 0.0 0.0 0.74''', # geometria: simbolo x y z (por linha)
6     basis='sto-3g',          # conjunto de funcoes de base
7     charge=0,                # carga total do sistema
8     spin=0,                  # numero de eletrons desemparelhados (2S)
9     unit='Angstrom',         # unidade das coordenadas (padrao)
10    verbose=4,               # nivel de detalhe da saida (0 a 9)
11 )
```

1.2.1 A geometria: o argumento `atom`

A geometria é fornecida como um texto em que cada linha traz um símbolo atômico seguido de suas coordenadas cartesianas x, y, z . Átomos podem ser separados por quebras de linha (como acima) ou por ponto e vírgula na mesma linha (`'H 0 0 0; H 0 0 0.74'`). As coordenadas estão em $\text{\AA}$ por padrão; para usar bohr, defina `unit='Bohr'`.

Atenção

O argumento `spin` no PySCF **não** é o spin total S nem a multiplicidade $2S + 1$: é o *número de elétrons desemparelhados*, $n_\alpha - n_\beta = 2S$. Assim, um sistema de camada fechada (como a H_2 no estado fundamental) tem `spin=0`; o átomo de hidrogênio, com um elétron solitário, tem `spin=1`; o átomo de silício, com dois elétrons $3p$ desemparelhados, tem `spin=2`. Errar esse número é a causa mais comum de falhas em cálculos de átomos e radicais.

1.2.2 Inspeccionando o objeto `Mole`

Uma vez construído, o objeto `mol` sabe muito sobre o sistema. Vale a pena interrogá-lo antes de calcular — é uma checagem barata que evita horas de frustração com um sistema mal definido:

Listing 1.3: Interrogando o objeto `Mole`.

```
1 from pyscf import gto
2
3 mol = gto.M(atom='H 0 0 0; H 0 0 0.74', basis='sto-3g')
4
5 print("Numero de eletrons:", mol.nelectron) # 2
6 print("Numero de atomos:", mol.natm)        # 2
```

```

7 print("Numero de funcoes de base (AOs):", mol.nao) # 2
8 print("Carga:", mol.charge, " Spin (2S):", mol.spin) # 0 0

```

```

Numero de eletrons: 2
Numero de atomos: 2
Numero de funcoes de base (AOs): 2
Carga: 0 Spin (2S): 0

```

O atributo `mol.nao` (*number of atomic orbitals*) informa quantas funções de base descrevem o sistema — um número que determina diretamente o custo do cálculo e a qualidade do resultado, como veremos a seguir.

1.3 O Que é uma Base?

Ao resolver a equação de Schrödinger num computador, não podemos representar os orbitais como funções contínuas arbitrárias. Em vez disso, escrevemos cada orbital molecular como uma **combinação linear** de funções conhecidas e fixas — as **funções de base**. O conjunto dessas funções é o *conjunto de base* (*basis set*).

Formalmente, um orbital molecular ψ_i é expandido como

$$\psi_i(\mathbf{r}) = \sum_{\mu=1}^N C_{\mu i} \phi_{\mu}(\mathbf{r}),$$

onde ϕ_{μ} são as funções de base (fixas) e $C_{\mu i}$ são os coeficientes que o cálculo determina. Quanto *maior* e mais flexível o conjunto de base (maior N), melhor a descrição — e maior o custo computacional. Encontrar o equilíbrio entre precisão e custo é uma arte central da química quântica.

Nota

Por que gaussianas? As funções de base do PySCF são *orbitais do tipo gaussiano* (GTOs), da forma $e^{-\alpha r^2}$. Elas não descrevem um átomo tão fielmente quanto as funções exponenciais $e^{-\alpha r}$ (do átomo de hidrogênio real), mas têm uma propriedade matemática preciosa: o produto de duas gaussianas centradas em pontos diferentes é outra gaussiana. Isso torna as integrais de dois elétrons — o gargalo do cálculo — tratáveis analiticamente. Para recuperar o comportamento correto, cada função de base é, na verdade, uma soma fixa (*contração*) de várias gaussianas.

1.3.1 Famílias de bases

Você encontrará algumas famílias de nomes recorrentes:

- **STO-3G** — uma base *mínima*: cada orbital atômico é aproximado por 3 gaussianas contraídas. É pequena, rápida e ótima para aprender e depurar, mas pouco precisa. Usaremos-la como “base de rascunho”.
- **Pople** (6-31G, 6-311G**) — bases de porte médio, clássicas na literatura.
- **correlação consistente** (cc-pVDZ, cc-pVTZ, cc-pVQZ) — bases sistemáticas, projetadas para convergir suave e previsivelmente rumo ao limite. “D”, “T”, “Q” indicam qualidade *double*, *triple*, *quadruple-zeta*.

- **def2** (def2-SVP, def2-TZVP) — família moderna e equilibrada de Karlsruhe, excelente para toda a tabela periódica — útil para os elementos mais pesados dos nossos nanocristais.

1.3.2 O efeito da base na prática

Nada ensina melhor do que os números. A Tabela 1.1 mostra a energia Hartree-Fock da molécula H_2 (à distância de 0,74 Å) calculada com bases crescentes. Observe duas tendências: o número de funções de base (**nao**) cresce, e a energia *desce* monotonamente, aproximando-se do limite Hartree-Fock ($\approx -1,1336$ Ha).

Tabela 1.1: Energia HF da H_2 ($R = 0,74$ Å) em função da base.

Base	Funções de base (nao)	Energia (Ha)
STO-3G	2	-1,116759
cc-pVDZ	10	-1,128700
cc-pVTZ	28	-1,132968
<i>limite Hartree-Fock</i>		$\approx -1,1336$

Dica

Uma energia mais baixa é sempre melhor? No método Hartree-Fock (e em qualquer método *variacional*), **sim**: o *princípio variacional* garante que a energia calculada é sempre um limite *superior* à energia verdadeira. Logo, entre duas bases, a que fornece a energia mais baixa é a mais próxima da realidade. Guarde esse princípio: ele é a bússola que orienta a escolha de bases e métodos.

1.4 Executando Hartree-Fock: o Objeto SCF

Com o sistema definido, escolhemos o método. O **Hartree-Fock** (HF) é o ponto de partida de quase toda a química quântica. Sua ideia central é uma aproximação de *campo médio*: em vez de tratar a repulsão instantânea entre cada par de elétrons — um problema intratável —, supõe-se que cada elétron se move no campo *médio* gerado por todos os outros. Como esse campo depende dos próprios orbitais que queremos encontrar, o problema é resolvido de forma **iterativa e autoconsistente**: daí o nome *Self-Consistent Field* (SCF). O Capítulo 4 detalhará o formalismo; por ora, basta saber que o PySCF faz todo o trabalho pesado.

O PySCF oferece três “sabores” de Hartree-Fock, e escolher o certo depende do spin do sistema:

- **scf.RHF** — *Restricted* HF: para sistemas de camada fechada (**spin=0**), em que todos os elétrons estão emparelhados. É o caso da H_2 .
- **scf.ROHF** — *Restricted Open-shell* HF: para sistemas com elétrons desemparelhados que se deseja tratar de forma restrita. É a escolha natural para átomos como H e Si.
- **scf.UHF** — *Unrestricted* HF: permite que elétrons de spin α e β ocupem orbitais espaciais diferentes. Essencial para radicais e para os cálculos de acoplamento de troca da Parte III.

Criado o objeto de método, o cálculo é disparado com **kernel()**, que devolve a energia total em hartree e a armazena também no atributo **mf.e_tot**:

Listing 1.4: Um cálculo Hartree-Fock completo da H₂.

```

1 from pyscf import gto, scf
2
3 mol = gto.M(atom='H 0 0 0; H 0 0 0.74', basis='sto-3g')
4
5 mf = scf.RHF(mol)
6 energia = mf.kernel()
7
8 print("Energia total: %.8f Ha" % energia)
9 print("Confirmando via atributo: %.8f Ha" % mf.e_tot)

```

```

converged SCF energy = -1.11675930739643
Energia total: -1.11675931 Ha
Confirmando via atributo: -1.11675931 Ha

```

A linha `converged SCF energy = ...` é impressa pelo próprio PySCF: ela anuncia que o ciclo autoconsistente convergiu e informa a energia final. Nossa molécula H₂ tem energia Hartree-Fock de $-1,11675931$ Ha.

1.5 Interpretando a Saída

Obter um número é fácil; *confiar* nele exige ler a saída com olhos críticos. Aumentando o nível de detalhe (`verbose=4`), o PySCF revela o andamento do ciclo SCF. Para tornar a leitura instrutiva, usemos a molécula de água, cujo SCF leva alguns ciclos para convergir (a H₂, minúscula, converge quase de imediato):

Listing 1.5: Acompanhando a convergência do SCF.

```

1 from pyscf import gto, scf
2
3 mol = gto.M(atom='''O 0.000 0.000 0.117
4               H 0.000 0.757 -0.469
5               H 0.000 -0.757 -0.469''',
6             basis='sto-3g', verbose=4)
7
8 mf = scf.RHF(mol)
9 mf.kernel()

```

Entre as muitas linhas da saída, as mais importantes são o registro dos ciclos:

```

cycle= 1 E= -74.9128050859 delta_E= -0.0755 |g|= 0.37 |ddm|= 1.69
cycle= 2 E= -74.9624186902 delta_E= -0.0496 |g|= 0.0424 |ddm|= 0.56
cycle= 3 E= -74.9629204358 delta_E= -0.000502 |g|= 0.00832 |ddm|= 0.0432
cycle= 4 E= -74.9629466547 delta_E= -2.62e-05 |g|= 6.37e-05 |ddm|= 0.015
cycle= 5 E= -74.9629466564 delta_E= -1.67e-09 |g|= 1.46e-05 |ddm|= 0.0001
cycle= 6 E= -74.9629466565 delta_E= -1.58e-10 |g|= 3.45e-06 |ddm|= 4.2e-05
converged SCF energy = -74.9629466565387

```

Cada linha é uma iteração do campo autoconsistente. Três colunas contam a história da convergência:

- E — a energia total naquele ciclo. Note-a *descendo* monotonamente e estabilizando.

- `delta_E` — a variação de energia em relação ao ciclo anterior. É o critério principal: quando cai abaixo de $\sim 10^{-9}$ Ha, o cálculo convergiu.
- `|g|` — a norma do gradiente (o quanto os orbitais ainda “querem” mudar). Também deve tender a zero.

Atenção

Convergiu $\neq$ correto. A mensagem `converged SCF energy` garante apenas que o algoritmo encontrou uma solução autoconsistente — não que ela seja a solução fisicamente desejada nem que a base seja adequada. E o pior cenário é o oposto: se aparecer `SCF not converged`, o número reportado **não tem valor** e deve ser descartado. Sempre confira se o cálculo convergiu antes de usar qualquer resultado.

1.5.1 Energias orbitais, ocupações e o *gap*

Além da energia total, o objeto de método guarda os **orbitais moleculares** resultantes. Dois atributos são especialmente úteis: `mf.mo_energy` (as energias dos orbitais, em ordem crescente) e `mf.mo_occ` (quantos elétrons ocupam cada orbital). Vejamos para a água:

Listing 1.6: Lendo orbitais, ocupações e o *gap* HOMO–LUMO.

```

1 import numpy as np
2
3 print("Energias orbitais (Ha):", np.round(mf.mo_energy, 4))
4 print("Ocupacoes:", mf.mo_occ)
5
6 # HOMO = ultimo orbital ocupado; LUMO = primeiro vazio
7 n_ocupados = int(mf.mo_occ.sum() // 2)      # 5 orbitais duplamente ocupados
8 homo = mf.mo_energy[n_ocupados - 1]
9 lumo = mf.mo_energy[n_ocupados]
10 gap = lumo - homo
11
12 print("HOMO: %.4f Ha   LUMO: %.4f Ha" % (homo, lumo))
13 print("Gap HOMO-LUMO: %.4f Ha = %.2f eV" % (gap, gap * 27.2114))

```

```

Energias orbitais (Ha): [-20.2418 -1.2684 -0.6179 -0.453  -0.3912  0.6056  0.7422]
Ocupacoes: [2.  2.  2.  2.  2.  0.  0.]
HOMO: -0.3912 Ha   LUMO: 0.6056 Ha
Gap HOMO-LUMO: 0.9968 Ha = 27.12 eV

```

A leitura é rica. A água tem 10 elétrons, distribuídos em 5 orbitais duplamente ocupados (ocupação 2.), enquanto os 2 orbitais restantes estão vazios (0.). O primeiro orbital, com energia profundíssima (−20,24 Ha), é essencialmente o caroço 1s do oxigênio. O HOMO (*Highest Occupied Molecular Orbital*) e o LUMO (*Lowest Unoccupied*) delimitam o *gap* — a energia mínima para excitar um elétron. Esse *gap* será uma das grandezas centrais do Projeto Âncora: no nanocristal de silício, é ele que traduz o confinamento quântico.

Dica

Quer todos os resultados de uma vez? O método `mf.analyze()` imprime um relatório completo — energias orbitais, ocupações, cargas atômicas e momentos de dipolo — num único comando. É uma forma rápida de “radiografar” um cálculo recém-concluído.

1.6 Mãos à Obra: do Átomo de Hidrogênio à Molécula H₂

Hora de calcular por conta própria. Faremos dois cálculos clássicos e compararemos os resultados com valores conhecidos — o hábito que separa o usuário crítico do apertador de botões.

Mãos à obra

Objetivo: calcular a energia do átomo de H e da molécula H₂, e observar como a energia do átomo de H se aproxima do valor exato (−0,5 Ha) à medida que a base melhora.

Parte A — O átomo de hidrogênio. Com um único elétron, o átomo de H é de camada aberta: use `spin=1` e `ROHF`.

```
1 from pyscf import gto, scf
2
3 for base in ['sto-3g', 'cc-pvdz', 'cc-pvtz']:
4     mol = gto.M(atom='H 0 0 0', basis=base, spin=1)
5     energia = scf.ROHF(mol).kernel()
6     print("%-8s -> %.6f Ha" % (base, energia))
```

```
sto-3g    -> -0.466582 Ha
cc-pvdz   -> -0.499278 Ha
cc-pvtz   -> -0.499810 Ha
```

Observe a energia subindo... digo, *descendo* rumo a −0,5 Ha, o valor exato. Com a base mínima STO-3G o erro é grande (~ 7%); com cc-pVTZ, cai para 0,04%. A química quântica computacional é, em boa medida, a arte de controlar esse tipo de erro.

Parte B — A molécula H₂. Camada fechada: `spin=0` e `RHF`.

```
1 from pyscf import gto, scf
2
3 mol = gto.M(atom='H 0 0 0; H 0 0 0.74', basis='cc-pvdz')
4 mf = scf.RHF(mol)
5 energia = mf.kernel()
6
7 print("Energia da H2: %.6f Ha" % energia)
8 print("HOMO: %.4f Ha" % mf.mo_energy[0])
```

```
converged SCF energy = -1.12870045...
Energia da H2: -1.128700 Ha
HOMO: -0.5924 Ha
```

Compare com a energia dos dois átomos de H isolados ($2 \times -0,499 = -0,998$ Ha na mesma base): a H₂ é bem *mais* estável, e a diferença é, essencialmente, a energia da ligação química.

Atenção

Camada fechada vs. aberta. Se você tentar rodar `scf.RHF` num átomo de H com `spin=1`, o PySCF reclamará — RHF pressupõe todos os elétrons emparelhados. E se esquecer o

`spin=1` (deixando o padrão `spin=0`) num sistema de número ímpar de elétrons, receberá um erro de consistência. Regra prática: **conte os elétrons desemparelhados e informe-os em `spin`.**

Projeto Âncora

Etapa 0 (conclusão) — O primeiro cálculo do átomo de silício.

O átomo de silício é o alicerce do nosso nanocristal. Antes de agrupá-lo às centenas, precisamos saber descrevê-lo isoladamente. Silício tem 14 elétrons e configuração $[\text{Ne}] 3s^2 3p^2$; os dois elétrons $3p$ ocupam orbitais distintos (estado fundamental 3P), logo há **dois elétrons desemparelhados**: `spin=2`.

Sua tarefa. No *notebook* `projeto_ancora_SiP.ipynb`, acrescente uma célula com o cálculo abaixo e execute-o.

```
1 from pyscf import gto, scf
2
3 # Atomo de silicio isolado: 14 eletrons, estado fundamental 3P (spin=2)
4 mol_si = gto.M(atom='Si 0 0 0', basis='sto-3g', spin=2)
5 print("Eletrons:", mol_si.nelectron, " AOs:", mol_si.nao)
6
7 mf_si = scf.ROHF(mol_si)
8 e_si = mf_si.kernel()
9 print("Energia do atomo de Si (STO-3G): %.6f Ha" % e_si)
```

```
Eletrons: 14 AOs: 9
converged SCF energy = -285.466211172452
Energia do atomo de Si (STO-3G): -285.466211 Ha
```

Repetindo com a base `def2-svp` (18 funções de base, mais precisa), a energia desce para $-288,749819$ Ha — coerente com o princípio variacional. Anote os dois valores no *notebook*.

Marco atingido. Com este cálculo, a Parte 0 está completa: você tem um ambiente funcional, domina o fluxo de trabalho do PySCF e executou seu primeiro cálculo do átomo que protagonizará todo o livro. Na Parte I, deixaremos de tratar um átomo isolado e começaremos a *construir o nanocristal*, agrupando dezenas de átomos de silício e inserindo o doador de fósforo.

Resumo do Capítulo

- Todo cálculo PySCF segue três passos: **definir o sistema** (`gto.M` → objeto `Mole`), **escolher o método** (`scf.RHF/ROHF/UHF`) e **executar** (`kernel()`).
- No `Mole`, a geometria vai no argumento `atom` (símbolo + $x y z$, em $\text{\AA}$ por padrão); `charge` é a carga e `spin` é o número de elétrons *desemparelhados* ($2S$), não a multiplicidade.
- Uma **base** expande os orbitais em funções gaussianas fixas. Bases maiores (`STO-3G` → `cc-pVDZ` → `cc-pVTZ`) dão energias menores e mais precisas, ao custo de mais funções (`nao`).
- Pelo **princípio variacional**, a energia HF é sempre um limite superior à energia verdadeira:

menor é melhor.

- Escolha **RHF** para camada fechada, **ROHF/UHF** para camada aberta (átomos, radicais).
- Sempre verifique a **convergência** (**converged SCF energy**) antes de confiar num resultado; leia **mo_energy** e **mo_occ** para obter HOMO, LUMO e o *gap*.
- **Projeto Âncora:** calculamos o átomo de silício isolado ($-285,466$ Ha em STO-3G), encerrando a Parte 0.

Exercícios

- 2.1 Camada fechada.** Calcule a energia Hartree-Fock do átomo de hélio (`atom='He 0 0 0'`, `spin=0`) com RHF nas bases STO-3G e cc-pVTZ. Compare com o valor de referência ($\approx -2,86$ Ha no limite HF) e comente qual base chega mais perto.
- 2.2 O papel do spin.** Tente calcular o átomo de hidrogênio com `scf.RHF` e `spin=0`. Leia a mensagem de erro e explique, em uma frase, por que ela ocorre. Em seguida, corrija usando `spin=1` e **ROHF**.
- 2.3 Curva de dissociação (esboço).** Usando um laço `for` sobre uma lista de distâncias H–H (por exemplo, 0,5 a 2,0 Å em passos de 0,25), calcule e imprima a energia RHF/STO-3G da H_2 em cada distância. Em qual distância a energia é mínima?
- 2.4 Inspeccionando orbitais.** Para a molécula de água da Listagem 1.5, imprima `mf.mo_occ` e confirme que há 5 orbitais ocupados. Calcule o *gap* HOMO–LUMO em eV.
- 2.5 Efeito da base no gap.** Calcule o *gap* HOMO–LUMO da H_2 nas bases STO-3G e cc-pVDZ. O *gap* aumenta ou diminui ao melhorar a base? Proponha uma explicação.
- 2.6 Projeto Âncora.** No seu *notebook*, calcule o átomo de silício (`spin=2`, **ROHF**) nas bases `sto-3g` e `def2-svp`. Registre numa célula de texto: (a) a energia em cada base; (b) o número de funções de base (`nao`); (c) por que a base maior fornece energia mais baixa.